

H1 — Functions

Oefening H1-A — Math Toolkit (basis)

Componenten: Input Group, Button Group, Placeholder

HTML

```
<section class="container py-4" id="h1-math-toolkit">
  <h3>H1 - Math Toolkit</h3>
  <div class="row g-3">
    <div class="col-md-6">
      <div class="input-group">
        <span class="input-group-text">A</span>
        <input id="mt_a" type="number" class="form-control"
placeholder="Getal A">
      </div>
    </div>
    <div class="col-md-6">
      <div class="input-group">
        <span class="input-group-text">B</span>
        <input id="mt_b" type="number" class="form-control"
placeholder="Getal B">
      </div>
    </div>

    <div class="col-12">
      <div class="btn-group" role="group">
        <button id="mt_add" class="btn btn-outline-primary">A + B</button>
        <button id="mt_sub" class="btn btn-outline-primary">A - B</button>
        <button id="mt_mul" class="btn btn-outline-primary">A × B</button>
        <button id="mt_div" class="btn btn-outline-primary">A ÷ B</button>
      </div>
    </div>

    <div class="col-12">
      <div id="mt_out" class="placeholder-glow mt-2">
        <span class="placeholder col-3"></span>
      </div>
    </div>
  </div>
</section>
```

Wat moet je doen (per element)

- #mt_a, #mt_b
 - Lees waarden met `Number()` bij elke klik op een operator-knop.
 - **Validatie:** als e´en van beide NaN is, toon in #mt_out: “Ongeldige invoer (vul beide getallen in)”.
- #mt_add, #mt_sub, #mt_mul, #mt_div
 - Koppel **click** events; roep per knop **een pure function** aan: `add(a,b)`, `sub(a,b)`, ...

- Bij delen: als $b \neq 0$, toon "Delen door 0 kan niet".
- #mt_out
 - Vervang de placeholder door platte tekst "Resultaat: X".
 - Gebruik **geen** alerts/cards; hou het in dit element.
 - Rond af op max 6 decimalen.

Tests

- 5 en 2 → 7, 3, 10, 2.5
- Leeg → foutmelding
- 10 en 0 → divisie: foutmelding

Oefening H1-B — Unit Converter

Componenten: Input Group, Button Group, Badges

HTML

```
<section class="container py-4" id="h1-unit-converter">
  <h3>H1 - Unit Converter</h3>
  <div class="row g-3">
    <div class="col-md-6">
      <div class="input-group">
        <span class="input-group-text">Centimeters</span>
        <input id="uc_cm" type="number" class="form-control" placeholder="bv.
180">
      </div>
    </div>
    <div class="col-md-6">
      <div class="btn-group">
        <button id="uc_to_m" class="btn btn-outline-secondary">→
meter</button>
        <button id="uc_to_in" class="btn btn-outline-secondary">→
inch</button>
      </div>
    </div>
    <div class="col-12">
      <span class="me-2">Meter:</span><span id="uc_m" class="badge text-bg-
secondary">-</span>
      <span class="ms-4 me-2">Inch:</span><span id="uc_in" class="badge text-
bg-secondary">-</span>
    </div>
  </div>
</section>
```

Wat moet je doen (per element)

- #uc_cm
 - Lees **bij knopklik** de waarde in cm.
 - Validatie op NaN of < 0 : verander beide badges naar "ongeldig".
- #uc_to_m, #uc_to_in
 - Click → bereken en vul **enkel** de betreffende badge bij.
 - $\text{cm} \rightarrow \text{m} = \text{cm} / 100$, $\text{cm} \rightarrow \text{inch} \approx \text{cm} / 2.54$.
 - Afronden op 2 decimalen.

- #uc_m, #uc_in
 - Update .textContent.
 - Geen layout-wijziging, enkel badge-tekst.

Tests

- 180 cm → 1.80 m, 70.87 inch
- Leeg of negatief → 'ongeldig'

H2 — Function Scoping

Oefening H2-A — Scope Lab

Componenten: Buttons, Badges, List Group, Accordion

```
<section class="container py-4" id="h2-scope-lab">
  <h3>H2 - Scope Lab</h3>

  <!-- Actieknoppen en status -->
  <div class="mb-3 d-flex align-items-center gap-2">
    <button id="sc_global" class="btn btn-secondary">Run global()</button>
    <button id="sc_local" class="btn btn-secondary">Run local()</button>

    <!-- Status badge -->
    <span id="sc_status" class="badge text-bg-secondary">
      Nog niet uitgevoerd
    </span>
  </div>

  <!-- Resultaat variabelenlijst -->
  <ul id="sc_vars" class="list-group mb-3"></ul>

  <!-- Accordion uitleg -->
  <div class="accordion" id="sc_acc">
    <div class="accordion-item">
      <h2 class="accordion-header" id="sc_acc_header">
        <button class="accordion-button" type="button" data-bs-
toggle="collapse" data-bs-target="#sc_acc_body">
          Function scope vs Block scope
        </button>
      </h2>
      <div id="sc_acc_body" class="accordion-collapse collapse show">
        <div class="accordion-body">
          In JavaScript is <code>var</code> function-scoped, wat betekent dat
          hij binnen de volledige functie blijft bestaan.
          <code>let</code> en <code>const</code> zijn block-scoped en bestaan
          enkel binnen de accolades waarin ze zijn gedeclareerd.
          Hoisting werkt anders voor var dan voor let/const, wat invloed
          heeft op bereikbaarheid.
        </div>
      </div>
    </div>
  </div>
```

</section>

Wat moet je doen (per element)

- #sc_global (Run global)
 - Roep global() aan: declareer variabelen met var, let, const op verschillende niveaus.
 - Toon in #sc_vars **welke variabelen bereikbaar zijn** (List Group items).
- #sc_local (Run local)
 - Roep local() aan: laat zien dat let/const block-scoped zijn.
 - Vergelijk reikwijdte: voeg regels toe in de lijst “zichtbaar / niet zichtbaar”.
- #sc_status
 - Zet badge op “global uitgevoerd” (grijs → blauw) of “local uitgevoerd”.
 - Wissel klas: text-bg-primary / text-bg-info.
- #sc_acc
 - Schrijf ze’lf toelichting (max 3 zinnen) over function vs block scope.

Tests

- Zorg dat bij beide runs de lijst verschilt (zichtbaarheid wijzigt).

Oefening H2-B — Shadowing Demo

Componenten: Input Group, List Group

HTML

```
<section class="container py-4" id="h2-shadowing">
  <h3>H2 - Variable Shadowing</h3>
  <div class="input-group mb-3">
    <span class="input-group-text">Basiswaarde</span>
    <input id="sh_base" type="number" class="form-control" placeholder="bv.
10">
    <button id="sh_run" class="btn btn-outline-dark">Run</button>
  </div>
  <ul id="sh_out" class="list-group"></ul>
</section>
```

Wat moet je doen (per element)

- #sh_base
 - Lees getal (default 10 indien leeg).
- #sh_run
 - Click → voer functie uit die binnen een inner block dezelfde naam herdefinieert (let x = ...).
 - Voeg 3 list-items toe in #sh_out:
 1. “buiten block: x = ...”
 2. “binnen block: x = ...”
 3. “na block: x = ...”

- #sh_out
 - Steeds **overschrijven** (geen accumulatie).

Tests

- Base 10 → toon drie verschillende regels die shadowing aantonen.

H3 — Named Functions

Oefening H3-A — Formatter Station

Componenten: Input Group, Button Group, List Group

```
<section class="container py-4" id="h3-formatter-station">
  <h3>H3 - Formatter Station</h3>

  <!-- Input Group -->
  <div class="input-group mb-3">
    <span class="input-group-text" id="fs_label">Tekst</span>
    <input
      id="fs_text"
      type="text"
      class="form-control"
      placeholder="Typ hier..."
      aria-describedby="fs_label"
    >
  </div>

  <!-- Button Group -->
  <div class="btn-group mb-3" role="group" aria-label="Formatter acties">
    <button id="fs_upper" type="button" class="btn btn-outline-dark">toUpperCase()</button>
    <button id="fs_lower" type="button" class="btn btn-outline-dark">toLowerCase()</button>
    <button id="fs_cap" type="button" class="btn btn-outline-dark">Capitalize()</button>
  </div>

  <!-- List Group: max 5 resultaten -->
  <ul id="fs_results" class="list-group"></ul>
</section>
```

Wat moet je doen (per element)

- #fs_text
 - Lees huidige tekst wanneer een knop wordt geklikt.
- #fs_upper, #fs_lower, #fs_cap
 - Koppel **click**; roep **named functions** aan: toUpper(str), toLower(str), capitalize(str).
 - capitalize: eerste letter hoofdletter, rest klein, trim() eerst.
- #fs_results
 - Voeg **bovenaan** een nieuw List Group item met resultaat.

- Beperk tot 5 resultaten: verwijder het laatste wanneer er >5 zijn.

Tests

- Lege input → voeg niets toe (geen fout, maar geen item).

Oefening H3-B — Slug Maker

Componenten: Floating Labels, Badge

HTML

```
<section class="container py-4" id="h3-slug-maker">
  <h3>H3 - Slug Maker</h3>
  <div class="form-floating mb-3">
    <input id="sm_title" class="form-control" placeholder="Titel">
    <label for="sm_title">Titel</label>
  </div>
  <div>
    <span class="me-2">Slug:</span>
    <span id="sm_slug" class="badge text-bg-secondary">-</span>
  </div>
</section>
```

Wat moet je doen (per element)

- #sm_title
 - **input** event → roep named function makeSlug(title) aan.
 - Regels: lowercase, spaties/underscores naar -, verwijder niet-alfa-numeriek behalve -, collapse meerdere -, trim koppeltekens.
- #sm_slug
 - Toon het resultaat live.

Tests

- " Hello__World!! " → hello-world

H4 — Functions & Strings (ES6)

Oefening H4-A — Template Previewer

Componenten: Floating Labels, Nav Pills, Placeholder

```
<section class="container py-4" id="h4-template-previewer">
  <h3>H4 - Template Previewer</h3>

  <!-- Inputs -->
  <div class="row g-3">
    <div class="col-md-4">
      <div class="form-floating">
        <input id="tp_first" class="form-control" placeholder="Voornaam">
        <label for="tp_first">Voornaam</label>
      </div>
    </div>
    <div class="col-md-4">
      <div class="form-floating">
```

```

        <input id="tp_last" class="form-control" placeholder="Achternaam">
        <label for="tp_last">Achternaam</label>
    </div>
</div>
<div class="col-md-4">
    <div class="form-floating">
        <input id="tp_city" class="form-control" placeholder="Stad">
        <label for="tp_city">Stad</label>
    </div>
</div>
</div>

<!-- Pills -->
<ul class="nav nav-pills my-3" role="tablist">
    <li class="nav-item" role="presentation">
        <button id="tp_p1" class="nav-link active" type="button" role="tab" aria
-selected="true">
            P1
        </button>
    </li>
    <li class="nav-item" role="presentation">
        <button id="tp_p2" class="nav-link" type="button" role="tab" aria-select
ed="false">
            P2
        </button>
    </li>
</ul>

<!-- Preview -->
<div class="card">
    <div class="card-body">
        <div id="tp_preview" class="placeholder-glow">
            <span class="placeholder col-6"></span>
            <span class="placeholder col-4"></span>
        </div>
    </div>
</div>
</section>

```

Wat moet je doen (per element)

- #tp_first, #tp_last, #tp_city
 - Lees waarden bij elke wijziging of preset-klik.
- #tp_p1, #tp_p2
 - Click → active pill class beheren.
 - Bepaal template met **template literals**, bv.:

- P1: Hallo \${first} \${last} uit \${city}!
 - P2: langere zin.
- #tp_preview
 - Vervang placeholder door de samengestelde tekst.
 - Lege velden → toon “Vul alle velden in”.

Tests

- Wisselen tussen pills past tekst direct aan.

Oefening H4-B — Initialen-generator

Componenten: Input Group, Badge

HTML

```
<section class="container py-4" id="h4-initials">
  <h3>H4 - Initialen Generator</h3>
  <div class="input-group mb-2">
    <span class="input-group-text">Volledige naam</span>
    <input id="ig_full" class="form-control" placeholder="bv. Tom Van Houtte">
    <button id="ig_make" class="btn btn-outline-secondary">Maak initialen</button>
  </div>
  <span id="ig_out" class="badge text-bg-secondary">-</span>
</section>
```

Wat moet je doen (per element)

- #ig_full
 - Lees naam; splits op spaties; filter lege stukjes.
- #ig_make
 - Click → initialen opbouwen via **string-functies**: eerste letter van elk deel in uppercase met punt (T.V.H.).
 - Minimaal 2 delen vereist; anders “onvolledige naam”.
- #ig_out
 - Toon resultaat.

Tests

- “tom van houten” → “T.V.H.”

H5 — ES6 Arrow Functions

Oefening H5-A — Counter Console

Componenten: Button Group, Progress, Badge

```
<section class="container py-4" id="h5-counter-console">
  <h3>H5 - Counter Console</h3>
```



```

<!-- Button Group -->
<div class="btn-group mb-3" role="group" aria-label="Counter controls">
  <button id="cc_inc1" type="button" class="btn btn-outline-
primary">+1</button>
  <button id="cc_inc5" type="button" class="btn btn-outline-
primary">+5</button>
  <button id="cc_dec1" type="button" class="btn btn-outline-
secondary">-1</button>
  <button id="cc_reset" type="button" class="btn btn-outline-
danger">Reset</button>
</div>

<!-- Badge showing current value -->
<span id="cc_val" class="badge text-bg-secondary">0</span>

<!-- Progress Bar -->
<div class="progress mt-3" style="height: 25px;">
  <div
    id="cc_bar"
    class="progress-bar"
    role="progressbar"
    style="width: 0%"
    aria-valuemin="0"
    aria-valuemax="100"
    aria-valuenow="0"
  >
    0%
  </div>
</div>
</section>

```

Wat moet je doen (per element)

- #cc_inc1, #cc_inc5, #cc_dec1, #cc_reset
 - Click → **arrow functions** die een module-scope count aanpassen.
 - Grenzen: 0–100.
- #cc_val
 - Toon huidig getal elke mutatie.
- #cc_bar
 - style.width = count + '%', aria-valuenow updaten en tekst X%.

Tests

- Reset zet alles terug op 0% en badge “0”.

Oefening H5-B — Inline Calculator (pijltjes)

Componenten: Input Group, Button Group

HTML

```

<section class="container py-4" id="h5-inline-calc">
  <h3>H5 - Inline Calculator</h3>
  <div class="input-group">

```

```

    <span class="input-group-text">N</span>
    <input id="ic_n" type="number" class="form-control" value="0">
    <button id="ic_plus" class="btn btn-outline-primary">+1</button>
    <button id="ic_minus" class="btn btn-outline-secondary">-1</button>
  </div>
</section>

```

Wat moet je doen (per element)

- #ic_plus, #ic_minus
 - Arrow functions die de **waarde in het inputveld** aanpassen (geen aparte variabele).
- #ic_n
 - Houd tussen -999 en 999 (clampen).

Tests

- Klikken verandert direct de inputwaarde.

H6 — ES6 Arrays

Oefening H6-A — Tags Manager

Componenten: Input Group, Button Group, List Group, Table, Badges

```

<section class="container py-4" id="h6-tags-manager">
  <h3>H6 - Tags Manager</h3>

  <!-- Input + Add -->
  <div class="input-group mb-3">
    <input id="tm_input" class="form-control" placeholder="Nieuwe tag">
    <button id="tm_add" class="btn btn-primary">Toevoegen</button>
  </div>

  <!-- Controls -->
  <div class="btn-group mb-3" role="group" aria-label="Tag acties">
    <button id="tm_sort" class="btn btn-outline-secondary">Sorteer A-Z</button>
  >
    <button id="tm_rev" class="btn btn-outline-secondary">Reverse</button>
    <button id="tm_clear" class="btn btn-outline-danger">Leegmaken</button>
  </div>

  <!-- List -->
  <ul id="tm_list" class="list-group mb-3">
    <!-- Dynamisch: <li class="list-group-item d-flex justify-content-between align-items-center">
      <span>tagnaam</span>
      <span class="badge text-bg-secondary rounded-pill">0</span>
    </li> -->
  </ul>

  <!-- Stats -->
  <div class="card">
    <div class="card-body p-0">
      <table class="table mb-0">

```

```

<tbody>
  <tr>
    <th scope="row" style="width: 40%;">Totaal aantal</th>
    <td><span id="tm_count">0</span></td>
  </tr>
  <tr>
    <th scope="row">Unieke tags</th>
    <td><span id="tm_unique">0</span></td>
  </tr>
</tbody>
</table>
</div>
</div>
</section>

```

Wat moet je doen (per element)

- #tm_input
 - trim(); geen lege of duplicate tags (case-insensitive).
- #tm_add
 - Push in array en **her-render** de List Group: elke tag als item met een **kleine badge** met index (0-based).
- #tm_sort, #tm_rev, #tm_clear
 - Sorteer A-Z, reverse, leeg array + UI.
- #tm_list
 - Volledig opnieuw opbouwen bij elke wijziging (geen append-rommel).
- #tm_count, #tm_unique
 - Count = lengte array; Unique = aantal unieke (case-insensitive set).

Tests

- ["JS", "js"] telt als 1 uniek.

Oefening H6-B — Shop Categories

Componenten: Breadcrumb, List Group (klikbare filters)

HTML

```
<section class="container py-4" id="h6-shop-cats">
  <h3>H6 - Shop Categories</h3>
  <nav aria-label="breadcrumb">
    <ol class="breadcrumb">
      <li class="breadcrumb-item active" id="sc_crumb">Alle</li>
    </ol>
  </nav>
  <ul id="sc_filters" class="list-group list-group-horizontal">
    <li class="list-group-item active">Alle</li>
    <li class="list-group-item">Laptops</li>
    <li class="list-group-item">Monitors</li>
    <li class="list-group-item">Accessoires</li>
  </ul>
  <ul id="sc_items" class="list-group mt-3"></ul>
</section>
```

Wat moet je doen (per element)

- #sc_filters
 - Click op een item → active state verplaatsen, filter dataset (local array) en render in #sc_items.
 - Update #sc_crumb naar de huidige categorie.
- #sc_items
 - Toon naam van product als list-item.
 - Geen kaarten/alerts.

Tests

- Wisselen van filter werkt instant en consistent met breadcrumb.

H7 — Nodes

Oefening H7-A — Node Inspector

Componenten: Buttons, List Group, Badges

```
<section
class="container py-4" id="h7-node-inspector">
  <h3>H7 - Node Inspector</h3>

  <p class="mb-2">
    Onderstaande structuur bevat zowel elementnodes als tekstnodes (spaties /
    line breaks).
    Vergelijk <code>childNodes</code> met <code>children</code>.
  </p>

  <!-- Container to inspect -->
  <div id="ni_container" class="border rounded p-3 mb-3">
    Tekst
    <ul>
      <li>Item 1</li>
      <li>
```

```

        Item 2
        <ul>
            <li>Subitem</li>
        </ul>
    </li>
    <li>Item 3</li>
</ul>
    Nog een tekstnode
</div>

<!-- Buttons -->
<div class="btn-group mb-3" role="group">
    <button id="ni_count_childNodes" type="button" class="btn btn-outline-
primary">
        Tel childNodes
    </button>
    <button id="ni_count_children" type="button" class="btn btn-outline-
primary">
        Tel children
    </button>
</div>

<!-- Results -->
<ul class="list-group">
    <li class="list-group-item d-flex justify-content-between align-items-
center">
        childNodes
        <span id="ni_out_nodes" class="badge text-bg-secondary">0</span>
    </li>
    <li class="list-group-item d-flex justify-content-between align-items-
center">
        children
        <span id="ni_out_children" class="badge text-bg-secondary">0</span>
    </li>
</ul>
</section>

```

Wat moet je doen (per element)

- #ni_container
 - Gebruik voor metingen: `childNodes.length` en `children.length`.
- #ni_count_childNodes, #ni_count_children
 - Click → toon respectieve aantallen in badges.
 - Zorg dat whitespace tekstnodes meetellen bij `childNodes`.
- #ni_out_nodes, #ni_out_children
 - Update badge-getal (geen extra text).

Tests

- Voeg extra spatie/linebreak toe en her-test `childNodes`.

Oefening H7-B — DOM Tree Flattener

Componenten: Collapse (toon 'ruwe HTML'), List Group

HTML

```
<section class="container py-4" id="h7-tree-flatten">
  <h3>H7 - DOM Tree Flattener</h3>
  <p>Toon alle elementnode-namen (depth-first) van het onderstaande blok.</p>
  <button class="btn btn-outline-secondary mb-2" data-bs-toggle="collapse"
data-bs-target="#tf_src">Toon bron</button>
  <pre id="tf_src" class="collapse">&lt;div id="tf_root"&gt;
    Tekst
    &lt;ul&gt;
      &lt;li&gt;A&lt;/li&gt;
      &lt;li&gt;B&lt;/li&gt;
    &lt;/ul&gt;
  &lt;/div&gt;</pre>

  <div id="tf_root" class="border rounded p-3">
    Tekst
    <ul><li>A</li><li>B</li></ul>
  </div>

  <ul id="tf_out" class="list-group mt-2"></ul>
</section>
```

Wat moet je doen (per element)

- #tf_root
 - Loop **recursief** door de **children** (ge'é'n textnodes), depth-first.
- #tf_out
 - Genereer list-items met de nodeName (bv. DIV, UL, LI, ...) in de volgorde van traversal.
 - Overschrijf bij elke run (zet het in een functie renderTree() en roep bij DOMContentLoaded).

Tests

- Als je extra toevoegt in de root, verschijnt die als eerste child.

H8 — Root Nodes

Oefening H8-A — Document Roots Dashboard

Componenten: Navbar, Table, Badges

```
<section class="container py-4" id="h8-document-roots">
  <h3>H8 - Document Roots Dashboard</h3>

  <!-- Navbar -->
  <nav class="navbar navbar-light bg-light border rounded px-3 mb-4">
    <span class="navbar-brand fw-semibold">DOM Root Info</span>
    <button id="rn_refresh" type="button" class="btn btn-outline-primary btn-sm ms-auto">
      Verversen
    </button>
```

```

</nav>

<!-- Table with badges -->
<table class="table table-bordered align-middle text-center">
  <thead class="table-light">
    <tr>
      <th>Node</th>
      <th>Naam</th>
      <th>Children count</th>
      <th>Heeft children?</th>
    </tr>
  </thead>
  <tbody>
    <!-- documentElement (HTML) -->
    <tr>
      <td><code>document.documentElement</code></td>
      <td><span id="rn_html_name" class="badge text-bg-secondary">-
</span></td>
      <td><span id="rn_html_children" class="badge text-bg-
secondary">0</span></td>
      <td><span id="rn_html_has" class="badge text-bg-
secondary">>false</span></td>
    </tr>
    <!-- head -->
    <tr>
      <td><code>document.head</code></td>
      <td><span id="rn_head_name" class="badge text-bg-secondary">-
</span></td>
      <td><span id="rn_head_children" class="badge text-bg-
secondary">0</span></td>
      <td><span id="rn_head_has" class="badge text-bg-
secondary">>false</span></td>
    </tr>
    <!-- body -->
    <tr>
      <td><code>document.body</code></td>
      <td><span id="rn_body_name" class="badge text-bg-secondary">-
</span></td>
      <td><span id="rn_body_children" class="badge text-bg-
secondary">0</span></td>
      <td><span id="rn_body_has" class="badge text-bg-
secondary">>false</span></td>
    </tr>
  </tbody>
</table>
</section>

```

Wat moet je doen (per element)

- #rn_refresh
 - Click → meet voor document.documentElement, document.head, document.body:
 - nodeName (bv. HTML, HEAD, BODY)
 - children.length

- `hasChildNodes()` (true/false string)
- Schrijf waarden in bijbehorende badges:
 - `#rn_html_name, #rn_html_children, #rn_html_has`
 - `#rn_head_name, #rn_head_children, #rn_head_has`
 - `#rn_body_name, #rn_body_children, #rn_body_has`
- Initieel bij **DOMContentLoaded** ook e'én keer verversen.

Tests

- Voeg een element in `<head>` (bv. meta) en check dat children stijgt.

Oefening H8-B — Section Pager

Componenten: Pagination, Offcanvas (navigatie)

HTML

```
<section class="container py-4" id="h8-section-pager">
  <h3>H8 - Section Pager</h3>

  <button class="btn btn-outline-secondary mb-2" data-bs-toggle="offcanvas"
data-bs-target="#sp_nav">Navigatie</button>
  <div class="offcanvas offcanvas-start" tabindex="-1" id="sp_nav">
    <div class="offcanvas-header">
      <h5 class="offcanvas-title">Ga naar sectie</h5>
      <button class="btn-close" data-bs-dismiss="offcanvas"></button>
    </div>
    <div class="offcanvas-body">
      <ul id="sp_list" class="list-group">
        <li class="list-group-item">Intro</li>
        <li class="list-group-item">Over</li>
        <li class="list-group-item">Contact</li>
      </ul>
    </div>
  </div>

  <nav>
    <ul class="pagination">
      <li class="page-item"><button id="sp_prev" class="page-
link">Vorige</button></li>
      <li class="page-item"><button id="sp_next" class="page-
link">Volgende</button></li>
    </ul>
  </nav>

  <div id="sp_sections">
    <section class="sp_sec" data-title="Intro">Dit is de intro.</section>
    <section class="sp_sec d-none" data-title="Over">Over ons.</section>
    <section class="sp_sec d-none" data-title="Contact">Contacteer
ons.</section>
  </div>
</section>
```


Wat moet je doen (per element)

- `.sp_sec` (meerdere)
 - Beheer **exact 1 zichtbare** sectie (d-none wisselen).
- `#sp_prev`, `#sp_next`
 - Click → navigeer naar vorige/volgende sectie (circuleer niet; disable knop aan randen met `.disabled` op `.page-item`).
- `#sp_list`
 - Klik op een list-item → spring naar overeenkomstige sectie op index.
 - Zet actieve item visueel (voeg `.active` aan het geklikte LI; verwijder van anderen).
- `#sp_nav`
 - Offcanvas sluit niet automatisch; navigatie mag ook met pijlen.

Tests

- Begin op sectie 0. Prev disabled; Next enabled.
- Klik "Contact" in offcanvas → spring correct.